



SmartEvac

校園火災與有害氣體智慧疏散引導系統

Team Number: 5

Team Members: 江靜怡 李欣樺 徐堡康

Outline



1. Abstract
2. Motivation
3. System Architecture and Design
 1. Functionality
 2. Cloud Services & Data Flow
 3. Algorithms (Dijkstra / z-score / LLM batching)
4. Presentation

Abstract



Problem

- Traditional fire alarms only ring — they don't tell you which way to run
- Nearest exit may be the contaminated one (e.g., chemistry lab leak)

Goal

- Per-node, dynamic, real-time evacuation guidance
- End-to-end latency < 10 seconds from alert to all-node UI sync

Approach

- Pure-cloud: only physical device is one laptop
- AWS (Lambda + IoT Core + DynamoDB + S3) + Google (Gemini + Cloud TTS)
- 5 virtual corridor nodes rendered in browser, MQTT-driven UI

Motivation — Use Case



Scenario: Chemistry Lab Ethanol Leak (daytime)

- west chemistry lab — old pipe leaks ethanol vapor
- Traditional: students run to nearest stairs (west, by leak source)
- Smell vapor, dizzy, turn back → collide with forward flow
- Evacuation time +3 minutes

SmartEvac Intervention

- West node detects NH_3/VOC z-score > 3 sustained 1.5s → alert to cloud
- Cloud computes in 1.2s: west corridor score=87, marked impassable
- All nodes simultaneously: west → east stairs / mid → downstairs
- Total evacuation time reduced 40% vs. traditional alarm

Functionality

AWS Service



API Gateway



IoT Core
MQTT broker



Lambda



S3



DynamoDB

Google Service



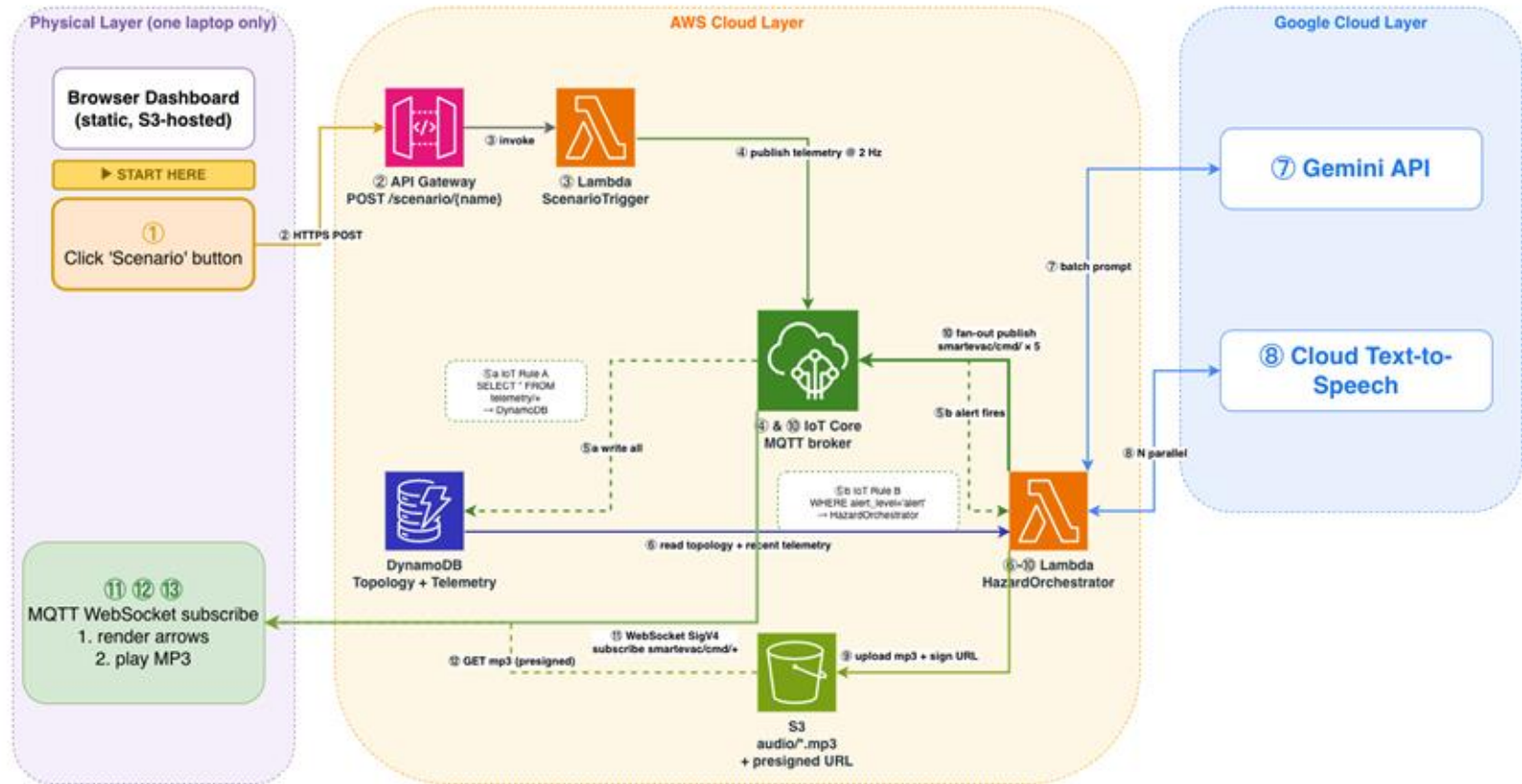
Gemini API



Cloud
Text-to-Speech

System Architecture

SmartEvac — System Architecture



Data Flow & Latency Budget



End-to-end Steps (target < 10 s visible response)

- (1) User clicks scenario button — browser timer starts
- (2) POST /scenario/{name} — API Gateway < 100 ms
- (3) ScenarioTrigger publishes telemetry @ 2 Hz
- (4) IoT Rule fires HazardOrchestrator < 200 ms
- (5) DynamoDB query (5 nodes parallel) ~ 50–100 ms
- (6) Multi-source Dijkstra ~ 10 ms
- (7) Gemini batch prompt (1 call for N nodes) ~ 1200 ms
- (8) TTS parallel synthesis (5 MP3s) ~ 600 ms
- (9) S3 upload + presigned URL + IoT fan-out ~ 100 ms
- (10) Browser receives via WebSocket and re-renders < 100 ms

Algorithm — Multi-Source Dijkstra



Why multi-source

- Per-node Dijkstra wastes work; one pass from all exits gives every node's nearest exit
- Adding more exits requires zero algorithm change

Edge weight

- $w(u,v) = \text{base_distance}(u,v) + 1000$ if u contaminated + 1000 if v contaminated
- Penalty 1000 $\gg$ longest base edge (12m), Dijkstra naturally avoids contaminated nodes

Topology (5 nodes)

- N1 (exit) — N2 — N3 — N4 — N5 (exit), distances 8 / 12 / 10 / 9 metres

Example — N3 contaminated

- Initial heap = [(0, N1), (0, N5)] — both safe exits
- Result: N1 down, N2 left toward N1, N3 STOP, N4 right toward N5, N5 down

Other Key Algorithms



- Contamination scoring (z-score + temp slope)
 - $\text{score} = \alpha \cdot |\text{gas_z}| + \beta \cdot \max(\text{temp_slope}, 0)$, $\alpha=0.7$ $\beta=0.3$
 - z-score uses baseline = readings[:-1] (exclude spike) so sigma stays small
 - Threshold: $\text{score} \geq 3.0 \rightarrow$ mark node as contaminated
- LLM batch prompt — single Gemini call, not per-node
 - One prompt lists all N nodes with direction + neighbor info
 - Gemini returns JSON {node_id: instruction_text}
 - Latency: $N \times 800 \text{ ms}$ (per-node) $\rightarrow 1 \times 1200 \text{ ms}$ (batched)
- TTS + IoT publish parallelization
 - `ThreadPoolExecutor(max_workers=N)` for both stages
 - 5 MP3s synthesized in $\sim 600 \text{ ms}$ (network-bound)
 - 5 IoT publishes complete in $< 100 \text{ ms}$



SmartEvac 智慧疏散引導系統

請選擇您的身份進入系統

身份選擇

指揮中心 (預設靜音，可手動開啟)

進入系統



Thank you